

Serveur Web Apache2

Installation, compréhension, hôtes virtuels, PHP (mod_php et PHP-FPM) et sécurisation d'un serveur Apache2 sous Debian.

PYB

Paul-Yann Bouvier

BTS SIO — option SISR

TYPE

Mission / TP système

ENVIRONNEMENT

VM Debian (VMware)

SERVICE

Apache 2.4 · PHP 7.4

OBJECTIFS

- Installer Apache2 et comprendre son arborescence
- Maîtriser les modules MPM et les processus
- Créer des hôtes virtuels (par nom, par IP, par port)
- Intégrer PHP et sécuriser le serveur (version, SSL, accès)

COMPÉTENCES MOBILISÉES

Apache2

VirtualHost

MPM event / prefork

PHP-FPM

SSL / TLS

htpasswd / .htaccess

SOMMAIRE

01 Introduction

02 Installation

03 Arborescence des fichiers

04 Processus et modules MPM

05 Répertoire et page d'accueil du site

06 Hôte virtuel par nom de domaine

07 Hôte virtuel par adresse IP

08 Changement du port d'écoute

09 PHP dans le site

10 Masquer la version et l'OS

11 Chiffrement SSL / TLS

12 Sécurisation des répertoires

1 Introduction

Apache est le nom de la fondation qui a développé le serveur Web **Apache httpd**, son premier projet, avant de nombreux autres. Présent depuis plus de 20 ans, c'est l'un des serveurs Web les plus utilisés au monde ; son principal concurrent est un autre logiciel libre, **Nginx**.

Site officiel <https://httpd.apache.org/>

Parts de marché <https://fr.hostadvice.com/marketshare/>

PARTIE A Installation et fonctionnement d'Apache2

2 Installation

2.1 Mettre à jour la machine et la renommer

```
TERMINAL (ROOT)
# apt update
# apt upgrade
# hostnamectl set-hostname apache2
```

2.2 Installer le paquet apache2

```
TERMINAL (ROOT)
# apt install apache2 -y
```

2.3 Tester l'installation

Depuis la machine physique, saisir l'adresse IP du serveur dans un navigateur : la page par défaut d'Apache doit s'afficher.

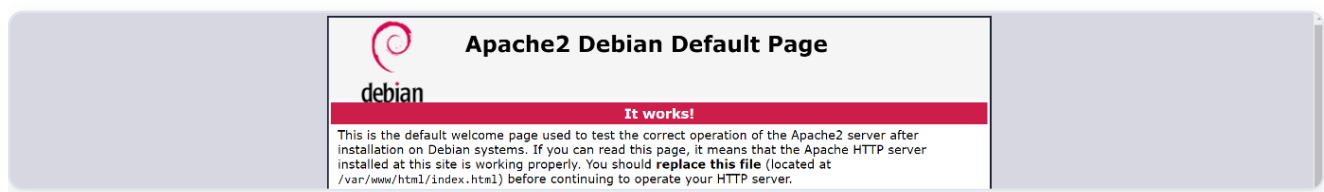


Figure 1 — Page « Apache2 Debian Default Page — It works! » sur <http://192.168.44.129>

3 Arborescence des fichiers

La commande `whereis` localise les répertoires utilisés par Apache2 :

TERMINAL (ROOT)

```
# whereis apache2
apache2: /usr/sbin/apache2 /usr/lib/apache2 /etc/apache2 /usr/share/apache2
/usr/share/man/man8/apache2.8.gz
```

Chemin	Rôle
/usr/sbin/apache2	L'exécutable (le répertoire <code>sbin</code> contient les binaires système).
/usr/lib/apache2	Les bibliothèques (modules).
/etc/apache2	Les fichiers de configuration.
/usr/share/apache2	Les fichiers partagés.
/usr/share/man/man8/apache2.8.gz	La documentation, consultable avec <code>man apache2</code> .

Le répertoire `/usr/sbin` contient aussi les scripts d'administration `a2enmod`, `a2dismod`, `a2ensite`, `a2dissite`, `a2enconf`, `a2disconf`, `a2query` et `apachectl`.

Le fichier `apache2.conf` décrit l'organisation de la configuration :

```
# It is split into several files forming the configuration hierarchy outlined
# below, all located in the /etc/apache2/ directory:
#
# /etc/apache2/
# -- apache2.conf
# -- ports.conf
# -- mods-enabled
# |   -- *.load
# |   -- *.conf
# -- conf-enabled
# |   -- *.conf
# -- sites-enabled
# |   -- *.conf
```

Figure 2 — Hiérarchie décrite dans `apache2.conf`

```
root@apache2:/etc/apache2# ls -al
total 88
drwxr-xr-x  8 root root 4096 18 sept. 20:06 .
drwxr-xr-x 78 root root 4096 18 sept. 20:24 ..
-rw-r--r--  1 root root 7224 12 août 13:51 apache2.conf
drwxr-xr-x  2 root root 4096 18 sept. 20:06 conf-available
drwxr-xr-x  2 root root 4096 18 sept. 20:06 conf-enabled
-rw-r--r--  1 root root 1782  8 août 2020 envvars
-rw-r--r--  1 root root 31063 8 août 2020 magic
drwxr-xr-x  2 root root 12288 18 sept. 20:06 mods-available
drwxr-xr-x  2 root root 4096 18 sept. 20:06 mods-enabled
-rw-r--r--  1 root root 320  8 août 2020 ports.conf
drwxr-xr-x  2 root root 4096 18 sept. 20:06 sites-available
drwxr-xr-x  2 root root 4096 18 sept. 20:06 sites-enabled
```

Figure 3 — Contenu de `/etc/apache2`

4 Processus et modules MPM

4.1 Observer les processus

TERMINAL (ROOT)

```
# ps aux --forest | grep apach[e]
```

Options : `a` (processus de tous les utilisateurs), `u` (propriétaire), `x` (processus non rattachés à un terminal).

```
root@apache2:~# ps aux --forest | grep apach[e]
root      2591  0.0  0.2  6628  4572 ?        Ss   10:32   0:00 /usr/sbin/apache2 -k start
www-data  2595  0.0  0.3  752748  6304 ?        Sl   10:32   0:00 \_ /usr/sbin/apache2 -k start
www-data  2596  0.0  0.5  752748  10384 ?       Sl   10:32   0:00 \_ /usr/sbin/apache2 -k start
```

Figure 4 — Un processus maître (root) et deux processus enfants (www-data)

- Le **processus maître** tourne sous `root`, état `Ss` : `S` = en attente d'un événement (*interruptible sleep*), `s` = leader de session.
- Les **processus enfants** tournent sous `www-data` pour des raisons de sécurité, état `Sl` : `l` = multi-thread. Ce sont eux qui répondent aux clients.

Lorsque la demande augmente, Apache crée de nouveaux processus enfants, chacun disposant de plusieurs threads. Un seul processus avec beaucoup de threads serait difficile à gérer, et de nombreux processus mono-thread consommeraient trop de mémoire : Apache adopte donc un fonctionnement **hybride**.

4.2 Les trois modules MPM

Les modules MPM (*Multi-Processing Modules*) déterminent la façon dont Apache gère les connexions. **Un seul MPM peut être actif à la fois** ; `mpm_event` est activé par défaut.

Module	Fonctionnement	Évaluation
mpm_prefork	Un processus enfant par connexion.	Fiable mais gourmand en mémoire : peu performant.
mpm_worker	Chaque processus enfant gère plusieurs connexions via des threads.	Performant, moins efficace avec des applications qui gèrent mal les threads.
mpm_event	Évolution de worker : un thread dédié gère les connexions <i>keep-alive</i> .	Le plus récent et le mieux optimisé (processus et mémoire).

Pour connaître le MPM chargé : lister `/etc/apache2/mods-enabled/` ou, plus simplement :

```
TERMINAL (ROOT)
# a2query -M
```

4.3 Changer de MPM manuellement (liens symboliques)

Les modules actifs sont des liens symboliques de `mods-available` vers `mods-enabled`. On désactive `mpm_event` puis on active `mpm_prefork` à la main :

```

TERMINAL (ROOT)
# cd /etc/apache2/mods-enabled/
# rm mpm_event.conf mpm_event.load
# service apache2 restart          # échoue : aucun MPM chargé
# ln -s ../mods-available/mpm_prefork.conf
# ln -s ../mods-available/mpm_prefork.load
# systemctl restart apache2
# ps aux --forest | grep apach[e]

```

```

(root@apache2)~[/etc/apache2/mods-enabled]
# ln -s ../mods-available/mpm_prefork.conf

(root@apache2)~[/etc/apache2/mods-enabled]
# ln -s ../mods-available/mpm_prefork.load

(root@apache2)~[/etc/apache2/mods-enabled]
# systemctl restart apache2

(root@apache2)~[/etc/apache2/mods-enabled]
# ps aux --forest | grep apach[e]
www-data 2685 0.0 0.0 3432 172 ? Ss 10:32 0:00 /usr/bin/htcacheclean -d 120 -p /var/cache/apach
00M -n
root 3550 0.0 0.2 6344 4296 ? Ss 11:27 0:00 /usr/sbin/apache2 -k start
www-data 3551 0.0 0.1 6604 3644 ? S 11:27 0:00 \_ /usr/sbin/apache2 -k start
www-data 3552 0.0 0.1 6604 3644 ? S 11:27 0:00 \_ /usr/sbin/apache2 -k start
www-data 3553 0.0 0.1 6604 3644 ? S 11:27 0:00 \_ /usr/sbin/apache2 -k start
www-data 3554 0.0 0.1 6604 3644 ? S 11:27 0:00 \_ /usr/sbin/apache2 -k start
www-data 3555 0.0 0.1 6604 3644 ? S 11:27 0:00 \_ /usr/sbin/apache2 -k start

```

Figure 5 — Avec prefork, Apache lance plusieurs processus enfants mono-thread (état S)

4.4 Changer de MPM avec les scripts

L'opération inverse (prefork → event) avec les scripts dédiés est plus simple et plus sûre :

```

TERMINAL (ROOT)
# a2dismod mpm_prefork
# a2enmod mpm_event
# systemctl restart apache2

```

```
(root@apache2)~# a2dismod mpm_prefork
Module mpm_prefork disabled.
To activate the new configuration, you need to run:
systemctl restart apache2

(root@apache2)~# systemctl restart apache2
Job for apache2.service failed because the control process exited with error code.
See "systemctl status apache2.service" and "journalctl -xe" for details.

(root@apache2)~# a2enmod mpm_event
Considering conflict mpm_worker for mpm_event:
Considering conflict mpm_prefork for mpm_event:
Enabling module mpm_event.
To activate the new configuration, you need to run:
systemctl restart apache2

(root@apache2)~# systemctl restart apache2

(root@apache2)~# ps aux --forest | grep apach[e]
www-data 2685 0.0 0.0 3432 172 ? Ss 10:32 0:00 /usr/bin/htcacheclean -d 120 -p /var/cache/apache2/
00M -n
root 3621 0.0 0.2 6628 4676 ? Ss 11:34 0:00 /usr/sbin/apache2 -k start
www-data 3622 0.0 0.3 752748 6316 ? Sl 11:34 0:00 \_ /usr/sbin/apache2 -k start
www-data 3623 0.0 0.3 752748 6316 ? Sl 11:34 0:00 \_ /usr/sbin/apache2 -k start
```

Figure 6 — Bascule vers mpm_event avec a2dismod / a2enmod

4.5 Déterminer le PID des processus

TROIS MÉTHODES

```
# pidof apache2
# cat /var/run/apache2/apache2.pid # PID du processus maître
# ps aux --forest | grep apach[e]
```

```
root@apache2:~# ps aux --forest | grep apach[e]
root 1554 0.0 0.3 11372 7416 ? Ss 10:41 0:00 /usr/sbin/apache2 -k start
www-data 1555 0.0 0.5 757624 11088 ? Sl 10:41 0:00 \_ /usr/sbin/apache2 -k start
www-data 1556 0.0 0.5 757624 11088 ? Sl 10:41 0:00 \_ /usr/sbin/apache2 -k start
```

Figure 7 — Les PID 1554 (maître), 1555 et 1556 (enfants)

PARTIE B Hébergement de sites — hôtes virtuels

Un serveur peut héberger plusieurs sites Web, soit grâce aux **hôtes virtuels** (*VirtualHost*) par nom, soit en affectant **une adresse IP par site**.

5 Répertoire et page d'accueil du site

TERMINAL (ROOT)

```
# mkdir /var/www/sitka
# cd /var/www && cp html/index.html sitka/
```

La page est ensuite personnalisée : titre d'onglet « **Sitka Alaska** » et message « bienvenue sur le site de sitka ».

6 Hôte virtuel par nom de domaine

6.1 Créer le fichier de configuration

TERMINAL (ROOT)

```
# cd /etc/apache2/sites-available
# cp 000-default.conf sitka.conf
# vim sitka.conf
```

/ETC/APACHE2/SITES-AVAILABLE/SITKA.CONF

```
<VirtualHost *:80>
    ServerName      www.sitka.local
    ServerAlias     *.sitka.local sitka.local
    ServerAdmin     admin@www.sitka.local
    DocumentRoot    /var/www/sitka

    ErrorLog        ${APACHE_LOG_DIR}/sitka_error.log
    CustomLog        ${APACHE_LOG_DIR}/sitka_access.log combined
</VirtualHost>
```

***:80** indique que l'hôte virtuel écoute sur le port 80 de toutes les interfaces.

Directive	Rôle
ServerName	Nom d'hôte pour lequel cette configuration s'applique (un seul par VirtualHost).

Directive	Rôle
ServerAlias	Noms supplémentaires répondant au même site.
ServerAdmin	Adresse de contact affichée sur certaines pages d'erreur.
DocumentRoot	Emplacement des fichiers du site. <i>Une configuration minimale peut s'arrêter ici.</i>
CustomLog	Journal d'accès dédié au site, au format <code>combined</code> .
ErrorLog	Journal d'erreurs dédié (format fixe).

Le bloc <Directory>

Il définit des règles propres à un répertoire. Options courantes : `ExecCGI` (scripts CGI autorisés), `FollowSymLinks` (suivi des liens symboliques, seule option active par défaut), `Includes` (inclusions côté serveur via `mod_include`), `Indexes` (liste des fichiers si aucun `index.html`, via `mod_autoindex`).
`AllowOverride None` interdit aux fichiers `.htaccess` de surcharger la configuration ; `AllowOverride All` les y autorise (ils sont alors prioritaires sur le VirtualHost).

6.2 Vérifier la syntaxe

```
TERMINAL (ROOT)
# apachectl -t
AH00558: apache2: Could not reliably determine the server's fully qualified domain name ...
Syntax OK
```

La syntaxe est correcte, mais Apache ne peut déterminer le FQDN du serveur. On le déclare dans un fichier de configuration dédié :

```
TERMINAL (ROOT)
# echo "ServerName apache2" > /etc/apache2/conf-available/fqdn.conf
# a2enconf fqdn
# apachectl -t
Syntax OK
```

6.3 Activer le site

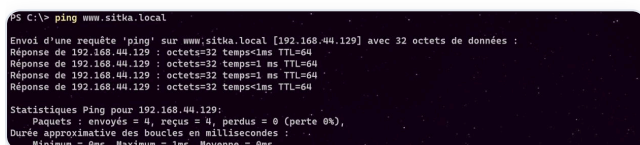
```
TERMINAL (ROOT)
# a2ensite sitka.conf
# systemctl reload apache2
```

6.4 Résolution de nom et test

En l'absence de serveur DNS, les noms sont déclarés dans le fichier `hosts` du client Windows (`C:\Windows\System32\drivers\etc\hosts`) et dans `/etc/hosts` du serveur. Sans cette déclaration, ni ping ni accès Web par le nom ne sont possibles.

/ETC/HOSTS (SERVEUR)

```
127.0.0.1    localhost
127.0.1.1    apache2
192.168.44.129 apache2
192.168.44.129 www.sitka.local
192.168.44.129 www.alaska.local
```



```
PS C:\> ping www.sitka.local

Envoi d'une requête 'ping' sur www.sitka.local [192.168.44.129] avec 32 octets de données :
Réponse de 192.168.44.129 : octets=32 temps<1ms TTL=64
Réponse de 192.168.44.129 : octets=32 temps<1 ms TTL=64
Réponse de 192.168.44.129 : octets=32 temps<1 ms TTL=64
Réponse de 192.168.44.129 : octets=32 temps<1ms TTL=64

Statistiques Ping pour 192.168.44.129:
    Paquets : envoyés = 4, reçus = 4, perdus = 0 (perte 0%),
    Durée approximative des boucles en millisecondes :
        Minimum = 0ms, Maximum = 1ms, Moyenne = 0ms
```

Figure 8 — Ping de `www.sitka.local` depuis le poste Windows



Figure 9 — `www.sitka.local` affiche la page « bienvenue sur le site de sitka »

7 Hôte virtuel par adresse IP

On ajoute deux adresses IP virtuelles, `192.168.44.160` et `192.168.44.161`, dans `/etc/network/interfaces` :

/ETC/NETWORK/INTERFACES

```
# The primary network interface
allow-hotplug ens33
iface ens33 inet dhcp

# ajout de cartes virtuelles
up ip addr add 192.168.44.160/24 dev ens33 label ens33:0
down ip addr del 192.168.44.160/24 dev ens33 label ens33:0
up ip addr add 192.168.44.161/24 dev ens33 label ens33:1
down ip addr del 192.168.44.161/24 dev ens33 label ens33:1
```

TERMINAL (ROOT)

```
# ifdown ens33 && ifup ens33
# ip ad
```

```
(root@apache2)~# ip ad
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 00:0c:29:8b:b6:95 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.44.129/24 brd 192.168.44.255 scope global dynamic ens33
        valid_lft 1735sec preferred_lft 1735sec
    inet 192.168.44.160/24 scope global secondary ens33:0
        valid_lft forever preferred_lft forever
    inet 192.168.44.161/24 scope global secondary ens33:1
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe8b:b695/64 scope link
        valid_lft forever preferred_lft forever
```

Figure 10 — Les adresses secondaires .160 et .161 sont actives sur ens33

Dans `sitka.conf`, Apache n'écoute plus sur toutes les adresses mais uniquement sur `192.168.44.160` :

```
SITKA.CONF
<VirtualHost 192.168.44.160:80>
    ...
</VirtualHost>
```

```
TERMINAL (ROOT)
# apachectl -t
# apache2ctl graceful
```

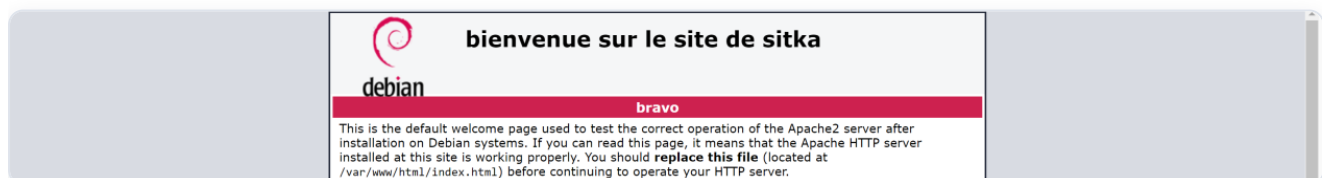


Figure 11 — Le site Sitka répond sur http://192.168.44.160

8 Changement du port d'écoute

Par défaut, Apache écoute sur le port 80 (HTTP) et 443 (HTTPS). On attribue au site le port **44** : déclaration dans `/etc/apache2/ports.conf`, puis dans le VirtualHost.

```
/ETC/APACHE2/PORTS.CONF
Listen 80
Listen 44

<IfModule ssl_module>
    Listen 443
</IfModule>
```

```
SITKA.CONF
<VirtualHost 192.168.44.160:44>
    ServerName www.sitka.local
    ...
</VirtualHost>
```

```
TERMINAL (ROOT)
# apachectl -t
# apache2ctl graceful
```

Le site est alors accessible à l'adresse `http://192.168.44.160:44`.

9 PHP dans le site

Deux méthodes permettent d'exécuter du PHP avec Apache : **mod_php** et **PHP-FPM**.

mod_php — inconvénients

Fonctionne uniquement avec Apache et impose `mpm_prefork` (un processus par requête). Apache et PHP partagent le même processus et la même mémoire, ce qui pose un risque de sécurité. Une seule version de PHP à la fois, un seul utilisateur (`www-data`).

PHP-FPM — avantages

Service indépendant d'Apache, utilisant le protocole FastCGI (supporté par de nombreux serveurs Web). Plusieurs versions de PHP et plusieurs pools, chacun avec son utilisateur, ses droits et ses dossiers. Sockets IPv4, IPv6 ou Unix. Adaptation dynamique du nombre de processus à la charge.

9.1 Mise en œuvre de mod_php

```
TERMINAL (ROOT)
# apt install libapache2-mod-php
```

En fin d'installation, un message signale que le paquet n'a pas pu basculer vers prefork ni activer PHP 7.4 (« *Could not switch to prefork MPM, not enabling PHP 7.4* »). Ces modules sont donc activés à la main :

```
TERMINAL (ROOT)
# a2dismod mpm_event
# a2enmod mpm_prefork
# a2enmod php7.4
# systemctl restart apache2
```

Page de test `/var/www/sitka/index.php` :

```
INDEX.PHP
<?php
phpinfo();
?>
```



PHP Version 7.4.25	
System	Linux apache2 5.10.0-9-amd64 #1 SMP Debian 5.10.70-1 (2021-09-30) x86_64
Build Date	Oct 23 2021 21:53:50
Server API	Apache 2.0 Handler
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/etc/php/7.4/apache2
Loaded Configuration File	/etc/php/7.4/apache2/php.ini
Scan this dir for additional .ini files	/etc/php/7.4/apache2/conf.d

Figure 12 — `phpinfo()` : Server API « Apache 2.0 Handler », configuration dans `/etc/php/7.4/apache2`

9.2 Mise en œuvre de PHP-FPM

On retire d'abord `mod_php`, puis on installe `php-fpm` :

```

TERMINAL (ROOT)
# apt purge libapache2-mod-php7.4 -y
# apt autoremove
# systemctl restart apache2
# apt install php-fpm -y

```

L'installation indique les étapes à suivre pour activer PHP-FPM dans Apache :

```

TERMINAL (ROOT)
# a2enmod proxy_fcgi setenvif
# a2enconf php7.4-fpm

```

Dans `sitka.conf`, on indique que tout fichier PHP est transmis au socket Unix de PHP-FPM :

```

SITKA.CONF
<FilesMatch \.php$>
    # 2.4.10+ can proxy to unix socket
    SetHandler "proxy:unix:/var/run/php/php7.4-fpm.sock|fcgi://localhost"
</FilesMatch>

```

PHP-FPM ne nécessite plus `prefork` : on revient à `mpm_event`.

```

TERMINAL (ROOT)
# a2query -M
prefork
# a2dismod mpm_prefork
# a2enmod mpm_event
# systemctl reload apache2
# a2query -m      # modules : setenvif, mpm_event, proxy_fcgi...
# a2query -c      # configurations : php7.4-fpm...

```

```
root@apache2:~# a2query -c
security (enabled by maintainer script)
server-cgi-bin (enabled by maintainer script)
sitka_name (enabled by site administrator)
charset (enabled by maintainer script)
localized-error-pages (enabled by maintainer script)
php7.4-fpm (enabled by site administrator)
other-vhosts-access-log (enabled by maintainer script)
```

Figure 13 — `a2query -c` : la configuration `php7.4-fpm` est active

PHP Version 7.4.25	
	
System	Linux apache2 5.10.0-9-amd64 #1 SMP Debian 5.10.70-1 (2021-09-30) x86_64
Build Date	Oct 23 2021 21:53:50
Server API	FPM/FastCGI
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/etc/php/7.4/fpm
Loaded Configuration File	/etc/php/7.4/fpm/php.ini
Scan this dir for additional .ini files	/etc/php/7.4/fpm/conf.d

Figure 14 — `phpinfo()` : Server API « FPM/FastCGI »

PARTIE C Sécurisation d'Apache2

10 Masquer la version et l'OS

Par défaut, Apache envoie des en-têtes HTTP contenant son nom, sa version et le système d'exploitation hôte. Divulguer ces informations facilite la recherche de vulnérabilités par un attaquant.

EN LOCAL

```
# apt-cache policy apache2
# apachectl -v
Server version: Apache/2.4.51 (Debian)
```

À DISTANCE, DEPUIS UNE MACHINE LINUX

```
# curl -I 192.168.44.129
HTTP/1.1 200 OK
Server: Apache/2.4.51 (Debian)
...
```

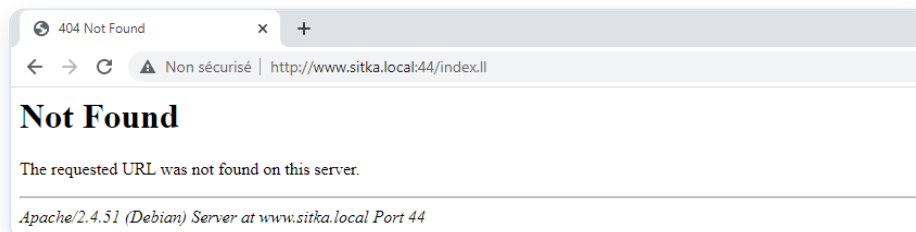


Figure 15 — Avant : la page d'erreur affiche « Apache/2.4.51 (Debian) Server at www.sitka.local Port 44 »

Les paramètres se règlent dans `/etc/apache2/conf-enabled/security.conf` (ou à la fin de `apache2.conf`) :

/ETC/APACHE2/CONF-ENABLED/SECURITY.CONF

```
ServerTokens    Prod    # au lieu de « OS » : en-tête réduit à « Server: Apache »
ServerSignature Off     # au lieu de « On » : plus de signature sur les pages d'erreur
```

TERMINAL (ROOT)

```
# systemctl reload apache2
```

Remarque

Commenter simplement la ligne `ServerTokens OS` ramène la directive à sa valeur par défaut (`Full`), qui est la plus bavarde : il faut donc la positionner explicitement à `Prod` .

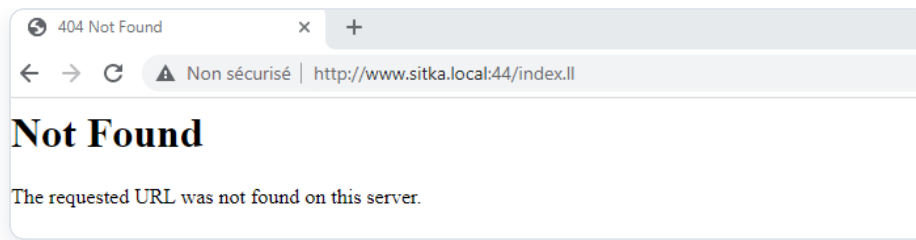


Figure 16 — Après : la page d'erreur n'affiche plus de signature serveur

11 Chiffrement SSL / TLS

11.1 Prérequis et module ssl

```

TERMINAL (ROOT)
# dpkg -l ssl-cert          # paquet présent : ii ssl-cert 1.0.39
# a2enmod ssl

```

11.2 Générer un certificat auto-signé

On crée un fichier **PEM** (*Privacy Enhanced Mail*) contenant un certificat auto-signé et sa clé privée :

```

TERMINAL (ROOT)
# make-ssl-cert /usr/share/ssl-cert/ssleay.cnf /etc/ssl/private/sitka.pem

```

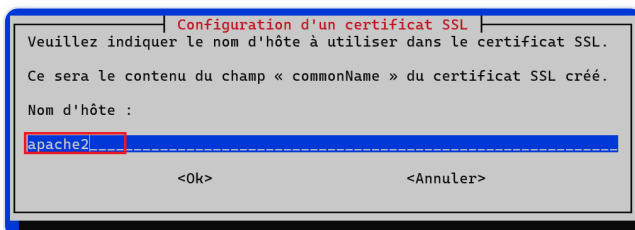


Figure 17 — commonName du certificat : apache2

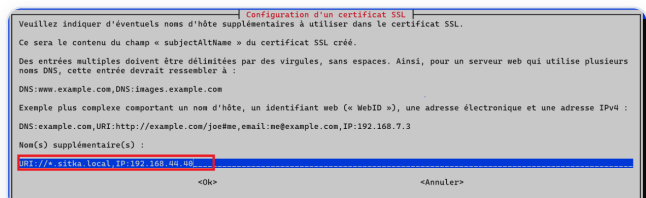


Figure 18 — subjectAltName : URI:*.sitka.local, IP:192.168.44.40

```

TERMINAL (ROOT)
# ls /etc/ssl/private
c8c3824d.0 sitka.pem ssl-cert-snakeoil.key
# cat /etc/ssl/private/sitka.pem
-----BEGIN PRIVATE KEY----- ... -----END PRIVATE KEY-----
-----BEGIN CERTIFICATE----- ... -----END CERTIFICATE-----

```

11.3 Hôte virtuel HTTPS

```
SITKA.CONF
<IfModule mod_ssl.c>
<VirtualHost _default_:443>
    ServerName      www.sitka.local
    ServerAlias      sitka.local *.sitka.local
    ServerAdmin      admin@sitka
    DocumentRoot     /var/www/sitka

    <FilesMatch \.php$>
        SetHandler "proxy:unix:/var/run/php/php7.4-fpm.sock|fcgi://localhost"
    </FilesMatch>

    ErrorLog         ${APACHE_LOG_DIR}/sitka-error.log
    CustomLog         ${APACHE_LOG_DIR}/sitka-acces.log combined

    SSLEngine on
    SSLCertificateFile /etc/ssl/private/sitka.pem
</VirtualHost>
</IfModule>
```

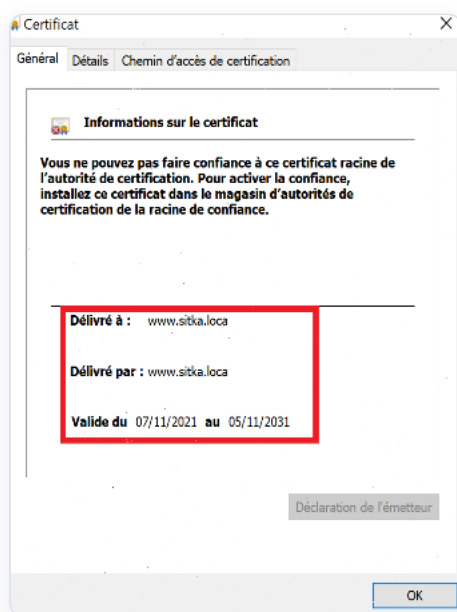


Figure 19 — Certificat présenté au navigateur (valable du 07/11/2021 au 05/11/2031)



Figure 20 — Accès en `https://www.sitka.local`

Certificat auto-signé

Le navigateur affiche « Non sécurisé » car le certificat n'est pas émis par une autorité reconnue. Le chiffrement est néanmoins actif ; en production, on utiliserait un certificat d'une autorité de certification (ex. Let's Encrypt).

12 Sécurité des répertoires

Deux méthodes permettent de restreindre l'accès à un répertoire :

Dans le VirtualHost (recommandé)

Meilleures performances ; nécessite des droits d'accès à la configuration du serveur.

Fichier .htaccess

Relu à chaque requête (impact sur les performances) ; utile pour déléguer certains réglages à des utilisateurs non root.

12.1 Création des comptes

TERMINAL (ROOT)

```
# htpasswd -c /etc/apache2/.password kaiser      # -c crée le fichier
# htpasswd /etc/apache2/.password cesar          # sans -c pour ne pas l'écraser
# cat /etc/apache2/.password
kaiser:$apr1$I2WvrU1g$ixe5wt6ZOY44nATtFDV8p/
cesar:$apr1$i0zRx0oi$ullh9Qw7WvqEQWU/waEwH1
```

Les mots de passe sont stockés sous forme de hachés (algorithme `apr1`, MD5 propre à Apache).

12.2 Méthode 1 : directive Directory dans le VirtualHost

SITKA.CONF

```
<Directory /var/www/sitka>
  AuthType Basic
  AuthName "utilisateur authentifié"
  AuthBasicProvider file
  AuthUserFile "/etc/apache2/.password"
  # Require ip 192.168.44.1
  Require valid-user
</Directory>
```

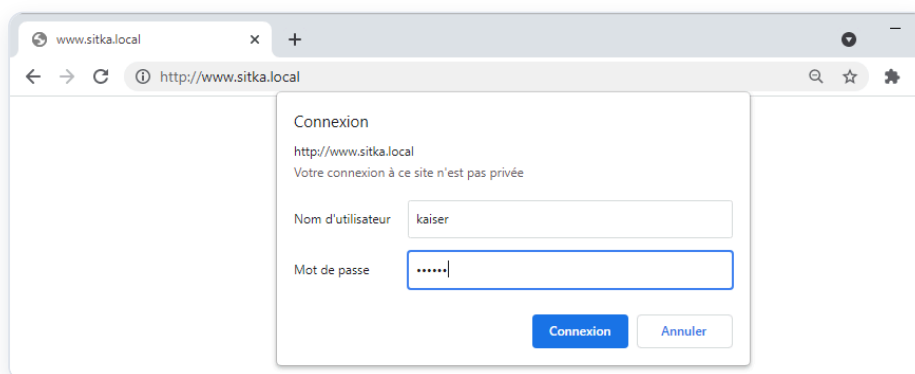


Figure 21 — Le navigateur demande une authentification (utilisateur kaiser)

12.3 Méthode 2 : fichier .htaccess

Il faut d'abord autoriser les surcharges dans `apache2.conf` en passant `AllowOverride None` à `All` pour `/var/www/` :

```
/ETC/APACHE2/APACHE2.CONF
<Directory /var/www/>
    Options Indexes FollowSymLinks
    AllowOverride All
    Require all granted
</Directory>
```

```
/VAR/WWW/SITKA/.HTACCESS
AuthType Basic
AuthName "Authentication user"
AuthUserFile /etc/apache2/.htpasswd
Require valid-user
```

Attention

`AuthUserFile` doit pointer vers le fichier réellement créé avec `htpasswd` (ici `/etc/apache2/.password`) ; sinon l'authentification échoue.

On recharge Apache sans couper les connexions en cours, puis on teste :

```
TERMINAL (ROOT)
# apachectl graceful
```

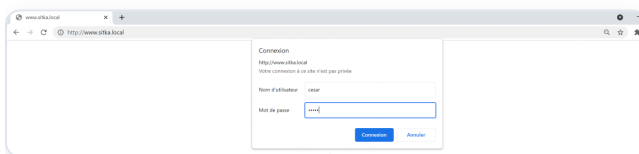


Figure 22 — Authentification avec l'utilisateur cesar

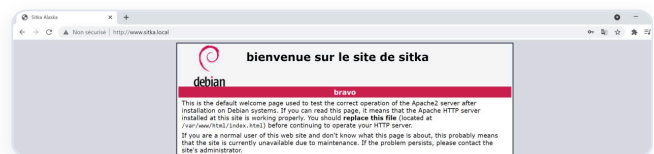


Figure 23 — Accès accordé : la page du site s'affiche

Bilan

Le serveur Apache2 héberge le site Sitka par nom, par IP et sur un port dédié, exécute PHP via PHP-FPM, chiffre les échanges en HTTPS, masque sa version et protège l'accès au site par authentification.